Time Series — Project Recipe Card

Stationarity · decomposition · ARIMA/ETS/Prophet/SARIMAX · walk-forward CV · multi-step · backtest. Deep companion: `time_series_cheatsheet.md` + `time_series.ipynb`

◆ Project recipe (11 steps)

1. **Frame** — horizon h, frequency, gap-to-features, exogenous vars, point vs probabilistic
2. **Chronological split FIRST** — train / val / test by TIME, never random
3. **Explore** — line plot, STL decomp, ACF/PACF, multiple seasonalities
4. **Stationarity tests** — ADF + KPSS together (regimes!)
5. **Feature engineering** — lags, rolling stats, calendar, exogenous (with proper `shift(1)`)
6. **Baseline trio** — Naive + Seasonal-Naive + Auto-ARIMA
7. **Model toolbox** — classical (ARIMA/ETS) → boosters on lags → DL last
8. **Multi-step strategy** — recursive / direct / multi-output (pick ONE)
9. **Walk-forward backtest** — expanding or rolling window, per-horizon metrics
10. **Diagnose** — Ljung-Box on residuals; per-horizon error decay
11. **Deploy** — refit on full data; roll forecasts; monitor drift

Time series breaks most ML assumptions. Random splits leak future into past; IID doesn't hold; residuals are autocorrelated. Treat as a separate discipline.

◆ Starter notebook skeleton

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from sklearn.model_selection import TimeSeriesSplit
np.random.seed(42)

df = df.sort_values("date").set_index("date").asfreq("D")  #
explicit freq!
df["y"] = df["y"].interpolate(method="time")               #
gaps

# 1. CHRONOLOGICAL split (NO shuffle)
n = len(df); cut1, cut2 = int(0.7*n), int(0.85*n)
train, val, test = df.iloc[:cut1], df.iloc[cut1:cut2],
df.iloc[cut2:]
print(train.index.min(), "→", train.index.max(), "→",
val.index.max(), "→", test.index.max())

# 2. Visual + decomposition
df["y"].plot(figsize=(12, 4))
stl = STL(train["y"], period=7).fit()    # daily data with
weekly cycle
stl.plot()

# 3. Stationarity
print("ADF p =", adfuller(train["y"].dropna())[1])    # <0.05 →
reject unit root
print("KPSS p =", kpss(train["y"].dropna())[1])       # >0.05 →
don't reject stationary

# 4. ACF / PACF for ARIMA order hints
plot_acf(train["y"].diff().dropna(), lags=40)
plot_pacf(train["y"].diff().dropna(), lags=40)
```

● Frame + setup

Frame checklist

- **Horizon h**: how far ahead (1 day? 4 weeks? 12 months?)
- **Frequency**: hourly / daily / weekly / monthly / irregular
- **Origin**: rolling (forecast issued daily) vs fixed
- **Features available at t+h**: known-future (calendar) vs unknown
- **Probabilistic vs point**: do you need P10/P50/P90?
- **Cold start**: new series with no history?

Frequency + index

```
df = df.sort_values("ts").set_index("ts")
df = df.asfreq("D")                    # B / D / H / W / MS / QS
# Or aggregate to regular grid:
df = df.resample("D").agg({"y": "sum", "x1": "mean"})
df["y"] = df["y"].interpolate("time")
assert df.index.is_monotonic_increasing
```

asfreq is essential — missing pandas freq breaks ACF/PACF/ARIMA.

Decomposition (STL)

```
from statsmodels.tsa.seasonal import STL, MSTL
stl = STL(y, period=7, robust=True).fit()    # weekly cycle in
daily data
stl.plot()
# Multiple seasonalities (e.g. daily + yearly):
mstl = MSTL(y, periods=(7, 365)).fit()
```

Stationarity (ADF + KPSS together)

ADF p	KPSS p	Verdict
<0.05	>0.05	STATIONARY ✓
>0.05	<0.05	NON-STATIONARY (unit root) → diff
<0.05	<0.05	trend-stationary → detrend
>0.05	>0.05	inconclusive → more data / transform

```
from statsmodels.tsa.stattools import adfuller, kpss
print("ADF  p =", adfuller(y.dropna())[1])
print("KPSS p =", kpss(y.dropna(), regression="c")[1])
```

● Feature engineering (lag-safe)

Lag features (golden rule: shift first!)

```
# For predicting y_t, you can use y_{t-1}, y_{t-7}, etc. — NEVER
y_t.
df["lag_1"]  = df["y"].shift(1)
df["lag_7"]  = df["y"].shift(7)
df["lag_28"] = df["y"].shift(28)

# Rolling features MUST shift first to avoid leakage:
df["roll_mean_7"]  = df["y"].shift(1).rolling(7).mean()
df["roll_std_28"]  = df["y"].shift(1).rolling(28).std()
# Expanding mean = "history up to t-1":
df["exp_mean"] = df["y"].shift(1).expanding().mean()
```

Most common bug: `df["y"].rolling(7).mean()` uses CURRENT value → leakage. Always `.shift(1).rolling()`.

Calendar features (always available)

```
df["dow"]   = df.index.dayofweek
df["month"] = df.index.month
df["doy"]   = df.index.dayofyear
df["is_weekend"] = df["dow"] >= 5
# Fourier features for smooth seasonality:
import numpy as np
for k in (1, 2, 3):
    df[f"sin_{k}"] = np.sin(2*np.pi*k*df["doy"]/365.25)
    df[f"cos_{k}"] = np.cos(2*np.pi*k*df["doy"]/365.25)
# Holidays:
import holidays
us = holidays.UnitedStates()
df["is_holiday"] = df.index.to_series().apply(lambda d: d in
us).astype(int)
```

Exogenous variables

- **Known-future** (calendar, scheduled promos, weather forecast): use the actual future value
- **Past-only** (lag of x): `x.shift(1)` only
- **Realtime stream** (weather/sensor): use the most recent available value at t

▲ Model toolbox

1 · Baselines (ALWAYS run these)

```
# Naive (last value):
y_hat_naive = train["y"].iloc[-1]
# Seasonal naive (last cycle):
y_hat_snaive = train["y"].iloc[-7:].values
# Drift (linear from first to last):
slope = (train["y"].iloc[-1] - train["y"].iloc[0]) / len(train)
y_hat_drift = train["y"].iloc[-1] + slope * np.arange(1, h+1)
```

If your model can't beat seasonal naive, it doesn't deserve to ship.

2 · ETS (exponential smoothing)

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
es = ExponentialSmoothing(
    train["y"], trend="add", seasonal="add", seasonal_periods=7,
    initialization_method="estimated").fit()
fc = es.forecast(h)
```

Holt-Winters. Cheap, robust, surprisingly hard to beat.

3 · ARIMA / SARIMA / SARIMAX

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
mod = SARIMAX(train["y"],
              exog=train[["x1","x2"]],
              order=(1, 1, 1),              # (p, d, q)
              seasonal_order=(1, 1, 1, 7))      # (P, D, Q, m)
res = mod.fit(disp=False)
fc  = res.get_forecast(steps=h, exog=test[["x1","x2"]])
mean, ci = fc.predicted_mean, fc.conf_int(alpha=0.20)

# Auto-select orders:
from pmdarima import auto_arima
auto = auto_arima(train["y"], seasonal=True, m=7,
                  start_p=0, max_p=3, start_q=0, max_q=3,
                  d=None, D=None,                   # let it
choose
                  stepwise=True, suppress_warnings=True)
```

4 · Prophet (Facebook) — fast, robust

```
from prophet import Prophet
m = Prophet(yearly_seasonality=True, weekly_seasonality=True,
            changepoint_prior_scale=0.05)
m.add_regressor("x1")
df_prophet = train.reset_index().rename(columns={"date":"ds",
"y":"y"})
m.fit(df_prophet)
future = m.make_future_dataframe(periods=h, freq="D")
future["x1"] = ...                          # exogenous values
fc = m.predict(future)
# fc["yhat"], fc["yhat_lower"], fc["yhat_upper"]
```

Use for: many seasonalities + known events + holidays. Don't tune for accuracy.

5 · LightGBM on lag features (THE workhorse)

```
import lightgbm as lgb
features = [c for c in df.columns if c != "y"]
X_tr, y_tr = train[features], train["y"]
X_va, y_va = val[features],   val["y"]
m = lgb.LGBMRegressor(
    n_estimators=2000, learning_rate=0.05,
    num_leaves=63, min_child_samples=20,
    subsample=0.8, colsample_bytree=0.8,
    n_jobs=-1, random_state=42)
m.fit(X_tr, y_tr,
      eval_set=[(X_va, y_va)],
      callbacks=[lgb.early_stopping(50)])
```

Add interaction/Fourier features. Often beats classical models with enough engineered lags.

6 · statsforecast (modern multi-series workhorse)

```
from statsforecast import StatsForecast
from statsforecast.models import AutoARIMA, AutoETS,
SeasonalNaive, Naive
# panel_df: columns [unique_id, ds, y]
sf = StatsForecast(
    models=[AutoARIMA(season_length=7),
AutoETS(season_length=7),
            SeasonalNaive(season_length=7), Naive()],
    freq="D", n_jobs=-1)
sf.fit(panel_df)
fc = sf.forecast(h=28, level=[80])    # 80% prediction intervals
# ~50× faster than pmdarima for the same auto-ARIMA, scales to
1M+ series.
```

For many related series. Companions: `neuralforecast` (DL), `hierarchicalforecast` (reconciliation).

7 · Deep learning (N-BEATS / TFT / DeepAR)

- **N-BEATS / N-HiTS**: pure backcast/forecast residual blocks; no seasonality assumption
- **TFT**: attention + interpretable; mixed-feature support
- **DeepAR**: probabilistic; many related series
- **Use only** when N_series > 100 AND classical/GBM plateau

```
from pytorch_forecasting import TemporalFusionTransformer
# Heavy machinery — refer to docs; not a tutorial-friendly path
```

Model picker

Situation	Start with
Single series, smooth	ETS (Holt-Winters)
Single series, seasonal	SARIMA(X) / Prophet
Many features, irregular	LightGBM on lags
Many related series	DeepAR / N-BEATS / global LGBM
Probabilistic intervals	SARIMAX (gaussian) / quantile LGBM
Multivariate VAR	statsmodels VAR / VECM

▲ Multi-step forecasting

3 strategies — pick ONE

Strategy	How	Trade-off
Recursive	predict t+1, feed in, predict t+2, …	error accumulates
Direct	one model per horizon (h models)	expensive; horizons inconsistent
Multi-output	single model outputs all h values	NN / multi-output GBM

```
# Recursive (sklearn style):
hist = train["y"].tolist()
preds = []
for _ in range(h):
    feats = build_features(hist)
    yh = m.predict(feats)[0]
    preds.append(yh)
    hist.append(yh)         # feed prediction back

# Direct (h independent models — align rows BEFORE fit):
models = {}
for k in range(1, h+1):
    y_k = y.shift(-k)
    mask = y_k.notna()
    models[k] = clone(base).fit(X[mask], y_k[mask])
```

◆ Metrics

Standard metrics

Metric	Formula	When
RMSE	√mean(e²)	default; penalises big errors
MAE	mean|e|	robust to outliers
MAPE	mean|e|/|y|	scale-free %; FAILS at y≈0
sMAPE	2|e|/(|y|+|ŷ|)	bounded; symmetric
MASE	MAE / MAE_naive	**RECOMMENDED** — scale-free, no /0
Pinball	quantile loss	probabilistic forecasts
CRPS	cont. ranked prob score	full distribution evaluation

Per-horizon reporting

```
# Report error PER HORIZON h=1, h=2, ..., h=H
errors = pd.DataFrame({"h": range(1, H+1),
                       "rmse": [rmse(y_true[:, h-1], y_pred[:,
h-1]) for h in range(1, H+1)]})
errors.plot(x="h", y="rmse", marker="o")
```

Average h=1..28 RMSE hides huge h=28 errors. Per-horizon shows decay.

MASE (the right default)

```
def mase(y_true, y_pred, y_train, m=1):
    """m=1 for non-seasonal, m=7 for weekly, m=12 for
monthly."""
    d = np.mean(np.abs(np.diff(y_train, n=m)))
    return np.mean(np.abs(y_true - y_pred)) / d
# MASE < 1 → beats naive; > 1 → worse than naive.
```

● Walk-forward validation (NEVER shuffle)

TimeSeriesSplit (sklearn)

```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5, test_size=30, gap=0)
for fold, (tr, te) in enumerate(tscv.split(X)):
    X_tr, y_tr = X.iloc[tr], y.iloc[tr]
    X_te, y_te = X.iloc[te], y.iloc[te]
    # Expanding window by default; rolling = pass
max_train_size=N
```

Train always precedes test in time. gap=h to prevent leakage of t+h features.

Walk-forward backtest loop

```
errors = []
origin = cut2                           # start at val/test
boundary
while origin + h <= len(df):
    train_slice = df.iloc[:origin]
    test_slice  = df.iloc[origin:origin+h]
    model = fit_model(train_slice)
    fc = model.forecast(h)
    errors.append(rmse(test_slice["y"], fc))
    origin += step                      # step=1 daily, step=h
non-overlapping
print(np.mean(errors), np.std(errors))
```

Expanding vs rolling

- **Expanding**: train grows; uses all history; vol-blind
- **Rolling**: fixed window; adapts to regime changes; loses early history
- **Default**: expanding when N_train small; rolling when N_train >> horizon

▲ Tuning

Grid / Optuna with TimeSeriesSplit

```
from sklearn.model_selection import GridSearchCV
gs = GridSearchCV(
    lgb_model, grid,
    cv=TimeSeriesSplit(n_splits=5, test_size=30),
    scoring="neg_root_mean_squared_error",
    n_jobs=-1, refit=True)
gs.fit(X_tr, y_tr)
```

SARIMAX order via auto_arima

```
from pmdarima import auto_arima
auto = auto_arima(train["y"], seasonal=True, m=7,
                  information_criterion="aicc",   # corrected
AIC
                  stepwise=True, n_jobs=-1)
print(auto.order, auto.seasonal_order)
```

ETS via AIC search

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
best, best_aic = None, np.inf
for trend in [None, "add"]:
    for seasonal in [None, "add", "mul"]:
        try:
            res = ExponentialSmoothing(y, trend=trend,
seasonal=seasonal,

seasonal_periods=7).fit()
            if res.aic < best_aic: best, best_aic = (trend,
seasonal), res.aic
        except: pass
```

◆ Residual diagnostics

Ljung-Box (autocorrelation in residuals)

```
from statsmodels.stats.diagnostic import acorr_ljungbox
lb = acorr_ljungbox(res.resid, lags=[10, 20, 30],
return_df=True)
print(lb)
# H0: residuals are white noise. Reject if p < 0.05 → model
leaves structure unexplained.
```

Standard diagnostics

- **Residual plot**: should look like white noise
- **Residual ACF**: no significant spikes (within 95% CI bands)
- **Q-Q**: residuals roughly Gaussian (intervals depend on it)
- **Heteroskedasticity**: residual^2 vs time; ARCH effects → use GARCH

```
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(res.resid, lags=40)
res.plot_diagnostics(figsize=(12, 8))    # SARIMAX has it built-
in
```

Uncertainty intervals

```
# Parametric (SARIMAX, ETS):
ci = res.get_forecast(h).conf_int(alpha=0.20)   # 80% PI
# Quantile (GBM):
lgb.LGBMRegressor(objective="quantile", alpha=0.1).fit(...)
# Conformal (distribution-free):
from mapie.regression import MapieTimeSeriesRegressor
mapie = MapieTimeSeriesRegressor(estimator=m, method="enbpi",
cv=tscv)
mapie.fit(X_tr, y_tr); y_p, y_int = mapie.predict(X_te,
alpha=0.10)
```

◆ Deploy

Roll forecasts in production

```
def daily_forecast(today: pd.Timestamp):
    history = load_history(end=today - pd.Timedelta("1D"))
    model = joblib.load("model.joblib")
    feats = build_features(history)
    fc = model.predict(feats)
    save_forecast(today, fc)
    return fc

# Schedule: cron daily, or Airflow / Prefect.
# Refit model: weekly or monthly (NOT every forecast).
```

Persistence

```
import joblib
# For statsmodels:
res.save("model.pkl")
# Load:
from statsmodels.tsa.statespace.sarimax import SARIMAXResults
res = SARIMAXResults.load("model.pkl")
# For sklearn/lgbm: joblib.dump(model, "model.joblib")
```

Monitoring

- **Actual-vs-forecast** dashboard, refresh daily
- **Rolling MASE** over last 28 days
- **Coverage check**: % of actuals inside 80% PI (should be 80%)
- **Re-train trigger**: rolling MASE > 1.0 OR coverage outside [70, 90]

▲ Top traps (6-month memory)

- **Random split** on temporal data → future leaks into past. Always chronological / `TimeSeriesSplit`.
- **Rolling without shift(1)** — `y.rolling(7).mean()` uses today. Use `y.shift(1).rolling(7).mean()`.
- **Centered rolling window** — peeks future. Always `center=False` (default).
- **Missing pandas freq** — `asfreq("D")` first; without it ACF/PACF/ARIMA misbehave.
- **Reporting only h=1 RMSE** — average hides multi-step error decay. Per-horizon table.
- **Timezone joins** — daylight saving silently drops/duplicates rows. Pin tz before merge.
- **ADF without KPSS** — single test misleads on trend-stationary series. Run both.
- **Skipping baselines** — if you can't beat seasonal naive, your model is noise.
- **Binary view of stationarity** — most real series are partly stationary. Diff if needed and re-test.

◆ Quick-reference (recall in 5 seconds)

Task	Canonical pattern
Index setup	`df.sort_values("ts").set_index("ts").asfreq("D")`
Split	chronological 70/15/15 by row count
Decompose	`STL(y, period=7, robust=True).fit().plot()`
Stationarity	`adfuller(y)[1]` AND `kpss(y)[1]`
Lag feature	`y.shift(1).rolling(7).mean()` (NEVER without shift)
Baseline	Naive + Seasonal-Naive + Auto-ARIMA
CV	`TimeSeriesSplit(n_splits=5, test_size=h, gap=0)`
Auto-order ARIMA	`pmdarima.auto_arima(y, seasonal=True, m=7)`
Best ML model	LightGBM on lag+calendar features
Metric default	MASE (then RMSE per-horizon)
Residual check	`acorr_ljungbox(resid, lags=[10,20,30])`
Intervals	SARIMAX `get_forecast(h).conf_int()` / quantile LGBM / MAPIE conformal